



KELOMPOK 2

Tugas Neural Network



ANGGOTA

1. ADRIAN AIFAJRI
2. KHARISMA MAULIDA
3. AUDI AULIA
4. MAULANA HAFIZH ARIPUTRA

INTRO DATA



```
✓ [1] import numpy as np
import pandas as pd
import os
import matplotlib.pyplot as plt
import seaborn as sns
```

```
✓ [2] df = pd.read_csv('Realestate_edit.csv')
df
```

	No	date	age	dist	store	lat	long	price	Village
0	1	2012	32.0	84.87882	10	24.98298	121.54024	37.9	Dapinglin
1	2	2012	19.5	306.59470	9	24.98034	121.53951	42.2	Dapinglin
2	3	2013	13.3	561.98450	5	24.98746	121.54391	47.3	Dapinglin
3	4	2013	13.3	561.98450	5	24.98746	121.54391	54.8	Dapinglin
4	5	2012	5.0	390.56840	5	24.97937	121.54245	43.1	Dapinglin
...	...	...	...	...	...	...	...	...	...
366	409	2013	18.5	2175.74400	3	24.96330	121.51243	28.1	New store
367	411	2012	5.6	90.45606	9	24.97433	121.54310	50.0	Dapinglin
368	412	2013	18.8	390.96960	7	24.97923	121.53986	40.6	Dapinglin
369	413	2013	8.1	104.81010	5	24.96674	121.54067	52.5	Dapinglin
370	414	2013	6.5	90.45606	9	24.97433	121.54310	63.9	Dapinglin

371 rows × 9 columns

INTRO DATA



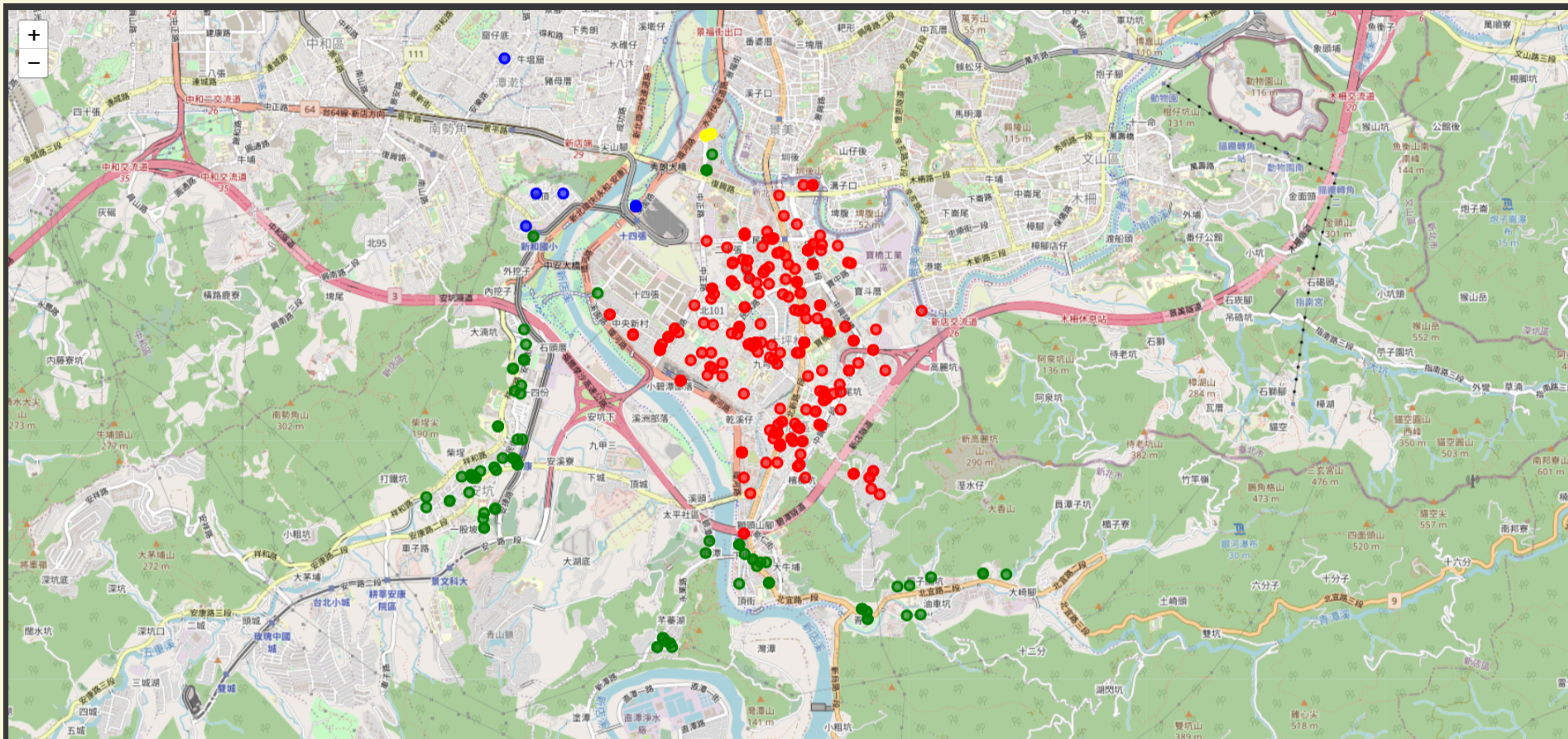
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 371 entries, 0 to 370  
Data columns (total 9 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
0   No          371 non-null   int64  
1   date        371 non-null   int64  
2   age         371 non-null   float64  
3   dist        371 non-null   float64  
4   store       371 non-null   int64  
5   lat         371 non-null   float64  
6   long        371 non-null   float64  
7   price       371 non-null   float64  
8   Village     371 non-null   object  
dtypes: float64(5), int64(3), object(1)  
memory usage: 26.2+ KB
```

```
[4] frekuensi_distrik = df['Village'].value_counts()  
print(frekuensi_distrik)
```

```
Dapinglin      258  
New store      101  
Pond            8  
Fulfilling      4  
Name: Village, dtype: int64
```


INTRO DATA





274 row: 5 column: 1

INTRO DATA



```
[14] df = df.drop(['lat', 'long'], axis=1)
```

```
dups=df.duplicated()  
print(dups.any())
```

```
False
```

```
df.columns
```

```
Index(['No', 'date', 'age', 'dist', 'store', 'price', 'Village'], dtype='object')
```

```
[ ] filtered_data = df[(df['price'] >= 100) & (df['price'] < 120)]  
  
print(filtered_data)
```

```
Empty DataFrame  
Columns: [No, date, age, dist, store, price, Village]  
Index: [ ]
```

```
[ ] filtered_data = df[(df['No'] == 357)]  
  
print(filtered_data)
```

	No	date	age	dist	store	price	Village	
	317	357	2012	10.3	211.4473	1	45.3	Dapinglin

INTRO DATA



```
[ ] df.describe()
```

	No	date	age	dist	store	price
count	371.000000	371.000000	371.000000	371.000000	371.000000	371.000000
mean	208.757412	2012.690027	17.475202	741.001846	4.493261	39.527763
std	120.433162	0.463107	11.742498	675.242324	2.780595	11.652456
min	1.000000	2012.000000	0.000000	23.382840	0.000000	7.600000
25%	104.500000	2012.000000	8.000000	279.172600	2.000000	30.650000
50%	210.000000	2013.000000	15.600000	461.784800	5.000000	40.000000
75%	313.000000	2013.000000	28.850000	1144.436000	7.000000	47.200000
max	414.000000	2013.000000	43.800000	3085.170000	10.000000	73.600000

INTRO DATA



```
[ ] df.head()
```

	No	date	age	dist	store	price	Village
0	1	2012	32.0	84.87882	10	37.9	Dapinglin
1	2	2012	19.5	306.59470	9	42.2	Dapinglin
2	3	2013	13.3	561.98450	5	47.3	Dapinglin
3	4	2013	13.3	561.98450	5	54.8	Dapinglin
4	5	2012	5.0	390.56840	5	43.1	Dapinglin

```
▶ dummies = pd.get_dummies(df.Village)  
dummies.head(3)
```



	Dapinglin	Fulfilling	New store	Pond
0	1	0	0	0
1	1	0	0	0
2	1	0	0	0

INTRO DATA



```
df2 = pd.concat([df,dummies],axis='columns')
df2.head()
```

	No	date	age	dist	store	price	Village	Dapinglin	Fulfilling	New store	Pond
0	1	2012	32.0	84.87882	10	37.9	Dapinglin	1	0	0	0
1	2	2012	19.5	306.59470	9	42.2	Dapinglin	1	0	0	0
2	3	2013	13.3	561.98450	5	47.3	Dapinglin	1	0	0	0
3	4	2013	13.3	561.98450	5	54.8	Dapinglin	1	0	0	0
4	5	2012	5.0	390.56840	5	43.1	Dapinglin	1	0	0	0

```
[ ] df3 = df2.drop('Village',axis='columns')
df3.head(2)
```

	No	date	age	dist	store	price	Dapinglin	Fulfilling	New store	Pond
0	1	2012	32.0	84.87882	10	37.9	1	0	0	0
1	2	2012	19.5	306.59470	9	42.2	1	0	0	0

INTRO DATA

```
[ ] df3.shape
```

```
(371, 10)
```

```
[ ] X1 = df3.drop(['price', 'No'], axis='columns')  
X1.head(3)
```

	date	age	dist	store	Dapinglin	Fulfilling	New store	Pond
0	2012	32.0	84.87882	10	1	0	0	0
1	2012	19.5	306.59470	9	1	0	0	0
2	2013	13.3	561.98450	5	1	0	0	0

```
[ ] y1 = df3.price  
y1.head(3)
```

```
0    37.9  
1    42.2  
2    47.3  
Name: price, dtype: float64
```



TEST LINEAR REGRESSION DF



```
[ ] from sklearn.model_selection import train_test_split
    from sklearn.linear_model import LinearRegression
    from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
    import pandas as pd

X1 = df3.drop(['price', 'No'], axis='columns')
y1 = df3.price

X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size=0.2, random_state=0)

# Buat model regresi linear
reg = LinearRegression()
reg.fit(X1_train, y1_train)

# Prediksi harga
y1_pred = reg.predict(X1_test)

# Evaluasi model
print('Linear Regression')
print('RMSE:', mean_squared_error(y1_test, y1_pred))
print('MAE:', mean_absolute_error(y1_test, y1_pred))
print('R^2:', r2_score(y1_test, y1_pred))
```

```
Linear Regression
RMSE: 69.57198179272213
MAE: 5.874841434851
R^2: 0.4959532800418943
```

TEST LINEAR REGRETION DF



df

	No	date	age	dist	store	price	Village
0	1	2012	32.0	84.87882	10	37.9	Dapinglin
1	2	2012	19.5	306.59470	9	42.2	Dapinglin
2	3	2013	13.3	561.98450	5	47.3	Dapinglin
3	4	2013	13.3	561.98450	5	54.8	Dapinglin
4	5	2012	5.0	390.56840	5	43.1	Dapinglin
...	...	...	...	...	...	...	...
366	409	2013	18.5	2175.74400	3	28.1	New store
367	411	2012	5.6	90.45606	9	50.0	Dapinglin
368	412	2013	18.8	390.96960	7	40.6	Dapinglin
369	413	2013	8.1	104.81010	5	52.5	Dapinglin
370	414	2013	6.5	90.45606	9	63.9	Dapinglin

371 rows × 7 columns

TEST LINEAR REGRETION DF



```
pred_df=pd.DataFrame({'Actual Value':y1_test,'Predicted Value':y1_pred,'Difference':y1_test-y1_pred})  
pred_df
```

	Actual Value	Predicted Value	Difference
141	35.6	41.558573	-5.958573
142	39.4	47.045343	-7.645343
313	25.7	17.451098	8.248902
60	36.2	41.595282	-5.395282
302	42.9	36.290331	6.609669
...	...	...	...
113	62.9	34.624182	28.275818
312	28.6	25.580647	3.019353
173	48.2	44.739707	3.460293
198	42.3	41.623656	0.676344
331	42.1	44.023933	-1.923933

75 rows × 3 columns

ABSTRAK



```
[66] def predict_price(village, date, age, dist, store):  
    # Mencari indeks lokasi dalam kolom X  
    loc_index = np.where(X1.columns == village)[0][0]  
  
    # Membuat vektor input dengan nilai awal 0  
    x = np.zeros(len(X1.columns))  
  
    # Menetapkan nilai fitur sesuai dengan argumen fungsi  
    x[0] = date  
    x[1] = age  
    x[2] = dist  
    x[3] = store  
  
    # Jika lokasi ditemukan dalam kolom X, atur nilai fitur sesuai indeks lokasi  
    if loc_index >= 0:  
        x[loc_index] = 1  
  
    # Lakukan prediksi harga menggunakan model regresi linear (reg)  
    predicted_price = reg.predict([x])[0]  
  
    return predicted_price
```

```
predicted_price = predict_price('Dapinglin', 2012, 32.0, 84.87882, 10)  
print(predicted_price)
```

45.280629460632554

/usr/local/lib/python3.10/dist-packages/sklearn/base.py:439: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
warnings.warn(

JSON



```
[ ] import json

# Menentukan path lengkap untuk file 'columns.json' di direktori '/content/drive/MyDrive/Model'
file_path = '/content/drive/MyDrive/Model/columns.json'

columns = {
    'data_columns': [col.lower() for col in X1.columns]
}

# Menyimpan data ke dalam file JSON di path yang sudah ditentukan
with open(file_path, "w") as f:
    f.write(json.dumps(columns))
```

JSON



```
[ ] import json

# Menentukan path lengkap untuk file 'columns.json' di direktori '/content/drive/MyDrive/Model'
file_path = '/content/drive/MyDrive/Model/columns.json'

columns = {
    'data_columns': [col.lower() for col in X1.columns]
}

# Menyimpan data ke dalam file JSON di path yang sudah ditentukan
with open(file_path, "w") as f:
    f.write(json.dumps(columns))
```

Real Estate

Transaction Date

2012



House Age

32

Convenience Store

10

Distance to MRT

85

Village

dapinglin



SELECT

Estimate Price

\$ 45.28 M



APLIKASI PREDIKSI